

Sumário

Planejamento Geral.....	2
Arquitetura Base: Híbrida.....	2
FASE 1: Versão 0.1 – Primeira Entrega Funcional (MVP)	2
FASE 2: Versão 1.0 – Expansão de Funcionalidades e RAG	3
FASE 3: Versão 1.5 – Segurança e Atualizações.....	4
FASE 4: Versão 2.0 – Integração com RPA.....	5
FASE 5: Versão 2.5 – Suporte a LLMs Locais (Offline).....	5
FASE 6: Versão 3.0 – Re-arquitetura com Agentes e MCPs.....	5
Plano Operacional Detalhado	6
FASE PRELIMINAR: Configuração e Preparação do Ambiente (Base).....	7
○ Objetivo PRE-0.1: Configurar Ambiente Firebase. [CONCLUÍDO]	7
○ Objetivo PRE-0.2: Configurar Ambiente de Desenvolvimento Local. [CONCLUÍDO]	7
○ Objetivo PRE-0.3: Validar a capacidade de empacotar e executar a aplicação em ambiente Windows alvo. [CONCLUÍDO]	7
○ Objetivo PRE-0.4: Implementar armazenamento seguro e criptografado da Chave de Serviço Firebase (Admin SDK JSON) localmente. [CONCLUÍDO]	8
○ Objetivo PRE-0.5: Implementar gerenciamento de configurações de proxy. [CONCLUÍDO]	8
○ Objetivo PRE-0.6: Desenvolver Utilitários Gerais. [Concluído Parcialmente]	9
○ Objetivo PRE-0.7: Estruturar a interface de comunicação com os serviços Firebase. [PENDENTE versão Client]	9
FASE 1: VERSÃO 0.1 – PRIMEIRA ENTREGA FUNCIONAL (MVP).....	10
○ Objetivo 1.1: Implementar pipeline completo de extração e análise de texto de PDFs.	10
○ Objetivo 1.2: Estabelecer sistema de logging local e em nuvem (específico do usuário)..	10
○ Objetivo 1.3: Implementar autenticação de usuário e gerenciamento seguro da chave API OpenAI do usuário.	11
○ Objetivo 1.4: Integrar com API OpenAI via LangChain usando um prompt único.	11
○ Objetivo 1.5: Implementar registro de métricas de uso e acerto da LLM (específicas do usuário).	11
○ Objetivo 1.6: Desenvolver a interface gráfica principal para interação com a análise.	12
○ Objetivo 1.7: Implementar funcionalidade de exportação simples para .docx.	12
○ Objetivo 1.8: Preparar a versão 0.1 para distribuição inicial.....	13
FASE 2: VERSÃO 1.0 – EXPANSÃO DE FUNCIONALIDADES E RAG.....	13
○ Objetivo 2.1: Suportar múltiplos LLMs, modelos e chaves de API.	13
FASE 3: VERSÃO 1.5 – SEGURANÇA E ATUALIZAÇÕES	15
FASE 4: VERSÃO 2.0 – INTEGRAÇÃO COM RPA	16

FASE 5: VERSÃO 2.5 – SUPORTE A LLMS LOCAIS (OFFLINE)	17
FASE 6: VERSÃO 3.0 – AGENTES E MODEL CONTEXT PROTOCOL (MCP)	18
Registros de Desenvolvimento	19
Troubleshooting Log	19
PROMPTs to Assistant AI	Erro! Indicador não definido.
Cronograma Semanal de Desenvolvimento	Erro! Indicador não definido.

Aplicação Python para Análise de Documentos PDF com IA

Planejamento Geral

Arquitetura Base: Híbrida

- **Componente Local:** Aplicação Python empacotada (via PyInstaller), executada na máquina Windows 10/11 do usuário. Utiliza Flet para servir uma interface web acessível via navegador em localhost. Responsável pelo processamento pesado (leitura de PDF, extração, chunking, embeddings v1.0, anonimização v1.5), chamadas diretas às APIs LLM e interação com o backend Firebase. Inclui um executável updater.exe separado para atualizações.
- **Componente Nuvem (Firebase):** Gerencia autenticação de usuários (Firebase Auth), armazena metadados, configurações, métricas e chaves de API LLM criptografadas (Firestore/RTDB), e hospeda logs enviados pela aplicação e os pacotes de atualização da aplicação (Firebase Storage).

FASE 1: Versão 0.1 – Primeira Entrega Funcional (MVP)

- **Objetivo Principal:** Entregar a funcionalidade básica de análise de PDFs com um LLM (OpenAI), interface gráfica local, sistema de logging, registro de métricas e exportação simples. Validar a arquitetura e o fluxo principal.
- **Entregas Chave:**
 - **Módulo pdf_processor:** Funcionalidade de extração de texto de PDFs.
 - **Integração com LangChain:** Utilização da biblioteca para interagir com a API da OpenAI.
 - **Prompt Único:** Implementação de um prompt geral, embutido e fixo para a análise.
 - **Sistema de Logging:**
 - Logging local em arquivos.
 - Logging em nuvem (Firebase Storage via SDK Cliente): Logs específicos da sessão do usuário são enviados para o Firebase Storage utilizando o SDK cliente do Firebase, autenticado com o token de ID do usuário. As Regras de Segurança do Storage garantirão que cada usuário só possa escrever em sua própria área de logs.
 - **Registro de Métricas (via SDK Cliente):**
 - Métricas de uso da aplicação e índice de acerto da LLM são armazenadas no Firestore utilizando o SDK cliente do Firebase, autenticado com o token de ID

do usuário. As Regras de Segurança do Firestore garantirão que cada usuário só possa escrever suas próprias métricas.

- Índice de acerto das respostas da LLM (comparando com edições do usuário antes da exportação), armazenado no Firestore.
- **Interface Gráfica (Flet):**
 - Interface para upload de PDF.
 - Interação com a API da OpenAI (envio de dados, exibição de resultados).
 - Autenticação de usuário (Firebase Auth via API REST do SDK Cliente): Utilização do FbManagerAuth para autenticar usuários e obter o Token de ID.
 - Configuração de chave API OpenAI do usuário (armazenada criptografada no Firestore via SDK Cliente):
 - Lógica local na aplicação cliente para criptografar a chave API OpenAI do usuário.
 - Utilização do SDK cliente do Firebase (autenticado com Token de ID) para salvar/recuperar a chave API OpenAI (já criptografada pela aplicação cliente) no Firestore, em um caminho acessível apenas pelo usuário autenticado (controlado por Regras de Segurança).
- **Funcionalidade de Exportação:** Exportação simples dos resultados em formato .docx.
- **Considerações Técnicas:**
 - Gerenciamento de Credenciais e Interação com Firebase:
 - `credentials_manager`: Utilizado exclusivamente para o gerenciamento seguro da chave de serviço do Firebase Admin SDK (localmente criptografada). O Admin SDK será usado para operações que não podem ser feitas pelo SDK cliente ou que exigem privilégios de backend (ex: verificar tokens de ID enviados pelo cliente, se necessário, ou para futuras funções de administração).
 - SDKs Cliente Firebase: A aplicação Python local utilizará os SDKs cliente do Firebase (ou APIs REST equivalentes que aceitem o Token de ID) para todas as interações com Firestore e Storage que envolvam dados do usuário (logs, métricas, chaves API LLM). A autenticação é feita via FbManagerAuth (que usa a API REST de Auth), e o Token de ID resultante é usado para as chamadas subsequentes aos serviços.
 - Regras de Segurança do Firebase: Serão cruciais para garantir que os usuários só possam acessar e modificar seus próprios dados no Firestore e Storage. Ex:
 - Firestore: `allow read, write: if request.auth.uid == userId;` (onde `userId` é parte do caminho do documento).
 - Storage: `allow write: if request.auth.uid == request.resource.metadata.ownerUid;` (ou similar, baseado no caminho).
 - Utilização do `config_manager` para configurações de proxy.

FASE 2: Versão 1.0 – Expansão de Funcionalidades e RAG

- **Objetivo Principal:** Aprimorar a flexibilidade do sistema com suporte a múltiplos LLMs, implementar RAG (Retrieval Augmented Generation) para melhor contextualização e oferecer opções de exportação customizáveis.

- **Entregas Chave:**
 - **Suporte a Múltiplos LLMs:**
 - Interface para selecionar diferentes LLMs e modelos.
 - Lógica para adaptar chamadas conforme o LLM/modelo escolhido.
 - **Suporte a Múltiplas Chaves de API:** Interface para gerenciar e selecionar diferentes chaves de API (armazenadas criptografadas no Firestore).
 - **Implementação de RAG:**
 - Geração de embeddings "on the fly" (localmente, na sessão do usuário) a partir do conteúdo do PDF.
 - Criação de índice vetorial em memória para busca de similaridade.
 - Integração com LangChain para enriquecer prompts com contexto relevante.
 - **Múltiplos Prompts com Chunking:**
 - Estratégia de divisão do texto (chunking).
 - Um prompt específico por atributo desejado para tabulação (configurável).
 - **Limites de Tokens Configuráveis:**
 - Interface para administrador (ou configuração no Firestore) definir limites por usuário.
 - Verificação e bloqueio na aplicação se o limite for excedido.
 - **Exportação em Templates Customizáveis:**
 - Suporte para templates .docx definidos pelo usuário (upload ou seleção).
 - Mapeamento dos resultados da LLM para placeholders no template.
 - **Interface Adicional "Chat with your PDF":**
 - Nova seção na UI para interação livre com o PDF, utilizando o RAG implementado.
- **Considerações Técnicas:**
 - Uso de bibliotecas como sentence-transformers e faiss-cpu (ou similar leve) para RAG.
 - Prompts e configurações de LLMs/modelos armazenados e gerenciados via Firestore para flexibilidade.

FASE 3: Versão 1.5 – Segurança e Atualizações

- **Objetivo Principal:** Incrementar a segurança dos dados processados e facilitar a distribuição de novas versões da aplicação.
- **Entregas Chave:**
 - **Sistema de Anonimização:**
 - Implementação de detecção de entidades nomeadas (NER) para identificar dados sensíveis (nomes, CPFs, etc.).
 - Mascaramento ou substituição dessas entidades antes do envio para a LLM.
 - Mapeamento para desanonimizar resultados, se necessário e seguro.
 - **Implementador/Atualizador Automático (Updater.exe):**
 - Desenvolvimento de um executável separado (updater.exe) para gerenciar atualizações.
 - Configuração do Firebase Storage para hospedar pacotes de atualização.
 - Lógica na aplicação principal para verificar novas versões, notificar e executar o updater.exe.
- **Considerações Técnicas:**
 - Uso de bibliotecas como spaCy ou Presidio para anonimização.

- O updater.exe deve lidar com download seguro, verificação de checksum (recomendado) e substituição de arquivos.

FASE 4: Versão 2.0 – Integração com RPA

- **Objetivo Principal:** Permitir que processos de RPA interajam com as funcionalidades da aplicação, automatizando fluxos de trabalho que envolvem análise de documentos.
- **Entregas Chave:**
 - **Integração com Scripts de RPA:**
 - Definição de uma interface de comunicação (ex: API local ou via Firebase Functions) para que scripts RPA possam enviar PDFs para análise e receber resultados.
 - Mecanismos de autenticação e autorização para os scripts RPA.
- **Considerações Técnicas:**
 - Avaliar a necessidade de expor uma API a partir do backend (Firebase ou um futuro servidor dedicado).
 - Definir contratos de dados e protocolos de comunicação com os scripts RPA.

FASE 5: Versão 2.5 – Suporte a LLMs Locais (Offline)

- **Objetivo Principal:** Oferecer a opção de processamento totalmente offline, utilizando LLMs executadas localmente na máquina do usuário (ou em um servidor na rede local).
- **Entregas Chave:**
 - **Integração com LLMs Locais:**
 - Suporte para download, configuração e execução de modelos de linguagem localmente.
 - Adaptação da lógica de chamada para direcionar para LLMs locais.
 - Interface para gerenciamento de modelos locais (seleção, status).
- **Considerações Técnicas:**
 - Avaliar requisitos de hardware do usuário (CPU, GPU, RAM).
 - Impacto no tamanho do pacote da aplicação e complexidade de distribuição/gerenciamento dos modelos.
 - Interface para seleção entre LLMs na nuvem e LLMs locais.

FASE 6: Versão 3.0 – Re-arquitetura com Agentes e MCPs

- **Objetivo Principal:** Evoluir a arquitetura da aplicação para um modelo mais sofisticado baseado em agentes de IA e o **Model Context Protocol (MCP)**, permitindo maior autonomia, flexibilidade e capacidade de lidar com tarefas mais complexas e interativas.
- **Entregas Chave:**
 - **Re-arquitetura Geral:**
 - Implementação de agentes de IA (ex: usando LangChain Agents) para orquestrar diferentes etapas do processo de análise.
 - Adoção e implementação do **Model Context Protocol (MCP)** para gerenciar a comunicação, o estado e o contexto entre os diferentes agentes e

componentes da aplicação, garantindo uma troca de informações padronizada e eficiente.

- **Considerações Técnicas:**

- Impacto significativo na estrutura do código e na lógica de negócio para acomodar o paradigma de agentes e o MCP.
- Novas necessidades de monitoramento e logging para rastrear o fluxo de contexto via MCP e o comportamento dos agentes.
- Gerenciamento de estado das interações e configurações dos agentes, em conformidade com o MCP.

Plano Operacional Detalhado

```
docs_analyzer_3/
├── .git/
├── logs/
│   └── *.log (Arquivos de log locais)
├── src/
│   ├── __init__.py
│   ├── config_manager.py      # (CONCLUÍDO - Base) Gerenciamento config de proxy / Keyring
│   ├── settings.py           # (CONCLUÍDO - Base) Configurações globais, chaves API (Web)
│   ├── utils.py              # (CONCLUÍDO - Base) Utilitários gerais, decoradores
│   ├── core/
│   │   ├── __init__.py
│   │   ├── ai_orchestrator.py # (PENDENTE - Integração LangChain v0.1)
│   │   ├── doc_generator.py   # (PENDENTE - Exportação DOCX v0.1)
│   │   └── pdf_processor.py    # (CONCLUÍDO - Base) Processamento, extração e análise
│   ├── database/
│   │   └── __init__.py        # (Não utilizado, pode ser para SQLAlchemy futuro)
│   ├── flet_ui/
│   │   ├── views/
│   │   │   ├── __init__.py
│   │   │   ├── login_view.py
│   │   │   ├── signup_view.py
│   │   │   └── home_view.py
│   │   ├── __init__.py
│   │   ├── app.py             # (CONCLUÍDO - Base) Configuração principal da App Flet
│   │   ├── components.py      # (CONCLUÍDO - Base) Componentes de UI reutilizáveis
│   │   ├── layout.py          # (CONCLUÍDO - Base) AppBar, NavigationRail, etc.
│   │   ├── router.py          # (CONCLUÍDO - Base) Lógica de roteamento de views
│   │   └── theme.py            # (CONCLUÍDO - Base) Definições de tema visual
│   ├── logger/
│   │   ├── __init__.py
│   │   └── logger.py          # (CONCLUÍDO - Base) Logging local e para Firebase Storage
│   ├── security/
│   │   ├── __init__.py
│   │   └── anonymizer.py      # (PENDENTE - Anonimização v1.5) Esqueleto para anonimização
│   ├── services/
│   │   ├── __init__.py
│   │   ├── credentials_manager.py # (CONCLUÍDO - Base) Gerencia chaves (serviço, API LLM)
│   │   └── firebase_manager.py   # (CONCLUÍDO - Base) Admin (AuthREST, Storage, Firestore)
│   └── tests/
│       ├── __init__.py
│       ├── conftest.py
│       ├── test_config_manager.py
│       └── services/
│           ├── __init__.py
│           └── test_credentials_manager.py
```

└─ .gitignore	
└─ pyproject.toml	
└─ poetry.lock	
└─ README.md	# (PENDENTE - Criação)
└─ run.py	# (CONCLUÍDO - Base) Script de inicialização da app Flet
Futuros Previstos:	
└─ updater/	# (v1.5) Para o código do updater.exe
└─ rpa_integration/	# (v2.0) Módulos para integração com RPA
└─ local_llms/	# (v2.5) Gerenciamento e interface para LLMs locais
└─ └─ models/	# (v2.5) (Potencialmente para modelos baixados)
└─ agents_pipelines/	# (v3.0) Módulos para a nova arquitetura de agentes e MCPs

FASE PRELIMINAR: Configuração e Preparação do Ambiente (Base)

Objetivo Principal: Estabelecer a fundação técnica, configurar serviços em nuvem (Firebase) e validar a viabilidade da execução local da aplicação empacotada.

- **Módulo: Configuração Firebase (Nuvem)**

- Objetivo PRE-0.1: Configurar Ambiente Firebase. [CONCLUÍDO]
 - Tarefa PRE-0.1.1: Criar Projeto Firebase no console.
 - Tarefa PRE-0.1.2: Habilitar Serviços: Authentication (Email/Senha), Firestore (modo Nativo), Storage.
 - Tarefa PRE-0.1.3: Definir Regras de Segurança Iniciais para Firestore e Storage (permitir acesso para desenvolvimento, a serem refinadas posteriormente).
 - Tarefa PRE-0.1.4: Gerar arquivo JSON de Credenciais de Serviço Firebase (Admin SDK).
- Marco de Teste PRE-0.1: Projeto Firebase acessível, serviços habilitados, regras básicas configuradas e arquivo JSON da chave de serviço baixado.

- **Módulo: Ambiente de Desenvolvimento Local**

- Objetivo PRE-0.2: Configurar Ambiente de Desenvolvimento Local. [CONCLUÍDO]
 - Tarefa PRE-0.2.1: Instalar Python 3.13+ e Poetry.
 - Tarefa PRE-0.2.2: Inicializar projeto Poetry e configurar pyproject.toml.
 - Tarefa PRE-0.2.3: Adicionar dependências base via Poetry (flet, firebase-admin, keyring, pyinstaller, requests, cryptography, etc. - conforme pyproject.toml).
 - Tarefa PRE-0.2.4: Configurar Git, criar .gitignore.
- Marco de Teste PRE-0.2: Ambiente local funcional, projeto Poetry inicializado com dependências, Git configurado e primeiro commit realizado.

- **Módulo: Teste de Viabilidade do Pacote (PyInstaller)**

- Objetivo PRE-0.3: Validar a capacidade de empacotar e executar a aplicação em ambiente Windows alvo. [CONCLUÍDO]
 - Tarefa PRE-0.3.1: Desenvolver uma aplicação mínima de teste com Flet (UI web local), integração Keyring para teste, inicialização básica do Firebase Admin SDK (simulada ou real com chave temporária) e uma chamada requests.get externa.
 - Tarefa PRE-0.3.2: Empacotar a aplicação mínima com PyInstaller (--onefile --windowed).
 - Tarefa PRE-0.3.3: Testar a execução do pacote em máquinas Windows 10/11 representativas do ambiente dos usuários, verificando abertura da UI Flet,

funcionamento do Keyring (se usado no teste), inicialização do Firebase e conectividade externa (considerando proxies).

- Marco de Teste PRE-0.3: Executável mínimo roda com sucesso no ambiente alvo, validando as tecnologias críticas.
- **Módulo: `src.services.credentials_manager` (Backend - Gerenciamento da Chave de Serviço Admin SDK)**
 - Objetivo PRE-0.4: Implementar armazenamento seguro e criptografado da Chave de Serviço Firebase (Admin SDK JSON) localmente. [CONCLUÍDO]
 - Tarefa PRE-0.4.1: Definir constantes para serviço Keyring (`KEYRING_SERVICE_FIREBASE`) e nome de usuário para a chave de criptografia Fernet (`KEYRING_USER_ENCRYPTION_KEY`).
 - Tarefa PRE-0.4.2: Implementar `get_encryption_key()` para buscar chave Fernet do Keyring.
 - Tarefa PRE-0.4.3: Implementar `_save_encryption_key()` para salvar chave Fernet no Keyring.
 - Tarefa PRE-0.4.4: Implementar `_generate_encryption_key()` para criar nova chave Fernet.
 - Tarefa PRE-0.4.5: Implementar `_encrypt_data()` e `_decrypt_data()` usando Fernet.
 - Tarefa PRE-0.4.6: Definir caminho para o arquivo criptografado da chave de serviço (`ENCRYPTED_SERVICE_KEY_PATH` em `APP_DATA_DIR`).
 - Tarefa PRE-0.4.7: Implementar `_save_encrypted_service_key()` para salvar o JSON da chave de serviço criptografado no arquivo `.enc`.
 - Tarefa PRE-0.4.8: Implementar `load_encrypted_service_key()` para carregar o arquivo `.enc`.
 - Tarefa PRE-0.4.9: Implementar `setup_credentials_from_file()` para orquestrar o processo de configuração inicial (solicitar JSON ao usuário, gerar chave Fernet, criptografar JSON, salvar chave Fernet no Keyring, salvar JSON criptografado no arquivo `.enc`).
 - Tarefa PRE-0.4.10: Implementar `get_decrypted_service_key_json()` para orquestrar a recuperação e descriptografia da chave de serviço JSON.
 - Tarefa PRE-0.4.11: Implementar `are_credentials_set_up()` para verificar se as credenciais do Admin SDK estão configuradas.
 - Marco de Teste PRE-0.4: Funções do `credentials_manager` permitem configurar, armazenar e recuperar a chave de serviço do Firebase Admin SDK de forma segura e criptografada, utilizando Keyring para a chave de criptografia. A aplicação consegue usar a chave recuperada para inicializar o Admin SDK.
- **Módulo: `src.config_manager` (Backend - Gerenciamento de Configurações Gerais)**
 - Objetivo PRE-0.5: Implementar gerenciamento de configurações de proxy. [CONCLUÍDO]
 - Tarefa PRE-0.5.1: Definir constantes Keyring para configurações de proxy (`PROXY_KEYRING_SERVICE`, etc.).
 - Tarefa PRE-0.5.2: Implementar `get_proxy_settings()` para recuperar configurações de proxy do Keyring (sem a senha).
 - Tarefa PRE-0.5.3: Implementar `save_proxy_settings()` para salvar configurações de proxy no Keyring, com tratamento seguro da senha do proxy (salvando-a separadamente ou deletando-a se não fornecida/desabilitada).
 - Marco de Teste PRE-0.5: Configurações de proxy podem ser salvas e recuperadas corretamente via Keyring.

- **Módulo: src.utils (Backend - Utilitários)**

- Objetivo PRE-0.6: Desenvolver Utilitários Gerais. [Concluído Parcialmente]
 - Tarefa PRE-0.6.1: Implementar decorador with_proxy() para aplicar configurações de proxy (lidas do config_manager) e, opcionalmente, desativar a verificação SSL para funções decoradas.
 - Tarefa PRE-0.6.2: Implementar count_tokens() e reduce_text_to_limit() (usando tiktoken).
- Marco de Teste PRE-0.6: Decorador with_proxy funciona conforme esperado, aplicando e revertendo configurações de proxy/SSL. Funções de tokenização operam corretamente.

- **Módulo: src.services.firebase_manager (Backend - Interface Firebase)**

- Objetivo PRE-0.7: Estruturar a interface de comunicação com os serviços Firebase. [PENDENTE versão Client]
 - Tarefa PRE-0.7.1: Implementar inicializar_firebase() que usa credentials_manager.get_decrypted_service_key_json() para configurar o Firebase Admin SDK. Nota: Este SDK será usado para operações limitadas de backend/administração, e não para interações de dados do usuário no dia-a-dia da aplicação cliente.
 - Tarefa PRE-0.7.2: Implementar FbManagerAuth para autenticação de usuário via API REST do Firebase Authentication, obtendo o Token de ID do Firebase.
 - Sub-tarefa: Método authenticate_user(email, password) retorna o Token de ID e informações do usuário.
 - Tarefa PRE-0.7.3: Adaptar/Implementar FbManagerStorage para operações com Firebase Storage utilizando SDKs cliente do Firebase (ou APIs REST com Token de ID).
 - Sub-tarefas: Métodos upload_text_user(user_token: str, text: str, storage_path: str), get_text_user(user_token: str, storage_path: str), etc. Estes métodos devem usar o user_token para autenticar a requisição e dependerão das Regras de Segurança do Storage para autorização. O storage_path deve incluir o userId para garantir isolamento.
 - Tarefa PRE-0.7.4: Adaptar/Implementar FbManagerFirestore para operações com Firestore utilizando SDKs cliente do Firebase (ou APIs REST com Token de ID).
 - Sub-tarefas: Métodos como save_user_api_key_client(user_token: str, user_id: str, service_name: str, encrypted_api_key: str), get_user_api_key_client(user_token: str, user_id: str, service_name: str), save_metrics_client(user_token: str, user_id: str, metric_data: dict), etc. Estes métodos devem usar o user_token e dependerão das Regras de Segurança do Firestore.
 - Tarefa PRE-0.7.5: Escrever e implantar Regras de Segurança robustas para Firestore e Storage que permitam aos usuários autenticados ler/escrever apenas seus próprios dados (baseado no request.auth.uid).
 - Marco de Teste PRE-0.7: FbManagerAuth autentica usuários e retorna Token de ID. Funções adaptadas em FbManagerStorage e FbManagerFirestore conseguem realizar operações CRUD nos respectivos serviços usando o Token de ID do usuário, e estas operações são corretamente permitidas/bloqueadas pelas Regras de Segurança.
-

FASE 1: VERSÃO 0.1 – PRIMEIRA ENTREGA FUNCIONAL (MVP)

Objetivo Principal da Fase: Entregar as funcionalidades core da primeira versão, incluindo processamento de PDF, integração com OpenAI via LangChain, interface Flet, logging, métricas e exportação DOCX.

- **Módulo: `src.core.pdf_processor` (Backend - Processamento de PDF) [CONCLUÍDO]**
 - Objetivo 1.1: Implementar pipeline completo de extração e análise de texto de PDFs.
 - Tarefa 1.1.1: Implementar estratégia de extração de texto PdfPlumberExtractor (incluindo `get_total_pages` e `extract_texts_from_pages`).
 - Tarefa 1.1.2: Implementar funções de pré-processamento de texto (`preprocess_text_basic`, `preprocess_text_advanced`).
 - Tarefa 1.1.3: Implementar utilitários de contagem de palavras e tokens (`count_unique_words`, `wrapper count_tokens`).
 - Tarefa 1.1.4: Implementar verificação de inteligibilidade do texto (`is_text_intelligible` usando `langdetect`).
 - Tarefa 1.1.5: Implementar funções de análise de similaridade (`analyze_text_similarity` com `sentence-transformers`) e relevância TF-IDF (`calculate_text_relevance_tfidf` com `sklearn`).
 - Tarefa 1.1.6: Implementar classe `PDFDocumentAnalyzer` orquestrando a extração, pré-processamento e análise de páginas (`analyze_pdf_document`).
 - Tarefa 1.1.7: Implementar método `filter_and_classify_pages` para identificar páginas relevantes e ininteligíveis.
 - Tarefa 1.1.8: Implementar método `group_texts_by_relevance_and_token_limit` para consolidar texto relevante.
 - Marco de Teste 1.1: Dado um PDF, o `PDFDocumentAnalyzer` consegue extrair texto, realizar análises (TF-IDF, similaridade), classificar páginas e agrupar texto relevante dentro de um limite de tokens. Os resultados são consistentes e as métricas (palavras, tokens) são calculadas.
- **Módulo: `src.logger.logger` (Backend - Logging) [CONCLUÍDO]**
 - Objetivo 1.2: Estabelecer sistema de logging local e em nuvem (específico do usuário).
 - Tarefa 1.2.1: Configurar `LoggerSetup` para logs locais rotacionados.
 - Tarefa 1.2.2: Configurar `LoggerSetup` para `RichHandler` no console.
 - Tarefa 1.2.3: Adaptar `CloudLogHandler` (ou criar um novo handler/lógica) para que, ao invés de usar o Admin SDK diretamente para o upload, ele prepare os logs e utilize o `FbManagerStorage.upload_text_user()` (da Task PRE-0.7.3), passando o Token de ID do usuário atual. O caminho no Storage deve ser específico do usuário (ex: `user_logs/{uid}/version/date.log`).
 - Sub-tarefa: Garantir que o Token de ID do usuário esteja acessível para o `CloudLogHandler` no momento do upload.
 - Sub-tarefa: Manter gerenciamento de buffer, thread, retries. O backup local em falha continua válido.
 - Marco de Teste 1.2: Logs locais e de console funcionam. Logs de nível INFO+ específicos da sessão do usuário são enviados para sua área designada no Firebase Storage usando o SDK cliente e Token de ID, respeitando as Regras de Segurança.
- **Módulo: `src.services.firebase_manager` e `src.flet_ui` (Autenticação e Gerenciamento de Chave API LLM)**

- Objetivo 1.3: Implementar autenticação de usuário e gerenciamento seguro da chave API OpenAI do usuário.
 - Tarefa 1.3.1 (UI): Criar view de Login em Flet.
 - Tarefa 1.3.2 (Backend): Conectar UI de Login ao `FbManagerAuth.authenticate_user()` para obter o Token de ID e uid.
 - Tarefa 1.3.3 (UI): Gerenciar estado de login e Token de ID na aplicação Flet.
 - Tarefa 1.3.4 (UI): Criar view/seção de Configurações para chave API OpenAI.
 - Tarefa 1.3.5 (Lógica Local Cliente): Implementar criptografia/descriptografia local da chave API OpenAI (Fernet com chave no Keyring do cliente ou derivada).
 - Tarefa 1.3.6 (Backend): Utilizar `FbManagerFirestore.save_user_api_key_client()` (da Task PRE-0.7.4) para salvar a chave API LLM (já criptografada localmente) no Firestore, usando o Token de ID do usuário para autenticar a escrita no caminho `user_api_keys/{uid}/{service_name}`.
 - Tarefa 1.3.7 (Backend): Utilizar `FbManagerFirestore.get_user_api_key_client()` para recuperar a chave API LLM criptografada do Firestore, usando o Token de ID do usuário.
 - Tarefa 1.3.8 (UI/Lógica): Conectar UI de Configurações, incluindo criptografia/descriptografia local e chamadas aos métodos cliente do `FbManagerFirestore`.
- Marco de Teste 1.3: Usuário autentica e obtém Token de ID. Chave API OpenAI é criptografada localmente, salva no Firestore (em `user_api_keys/{uid}/...`) usando o Token de ID, e pode ser recuperada e descriptografada. Regras de Segurança impedem acesso/modificação por outros usuários.
- **Módulo: `src.core.ai_orchestrator` (Backend - Lógica de Negócio IA) & LangChain**
 - Objetivo 1.4: Integrar com API OpenAI via LangChain usando um prompt único.
 - Tarefa 1.4.1: Adicionar langchain e openai ao `pyproject.toml`.
 - Tarefa 1.4.2: Definir um prompt único e geral, embutido na aplicação, para a tarefa de análise de documentos.
 - Tarefa 1.4.3: No `ai_orchestrator.py` (ou módulo similar), implementar função que:
 - Recebe o texto processado pelo `PDFDocumentAnalyzer`.
 - Recupera a chave API OpenAI do usuário (descriptografada - via lógica da Task 1.3).
 - Usa LangChain para construir a chamada à API OpenAI com o prompt definido e o texto.
 - Trata respostas e erros da API.
 - Marco de Teste 1.4: Aplicação consegue enviar o texto extraído de um PDF (após processamento) para a API da OpenAI usando LangChain e o prompt fixo, e recebe uma resposta. Erros de API são capturados.
- **Módulo: `src.services.firebase_manager` e `src.flet_ui` (Backend & Frontend - Métricas)**
 - Objetivo 1.5: Implementar registro de métricas de uso e acerto da LLM (específicas do usuário).
 - Tarefa 1.5.1: Definir estrutura de dados para métricas no Firestore (ex: coleção `user_metrics/{uid}/metrics/{docId}`).

- Tarefa 1.5.2: Utilizar `FbManagerFirestore.save_metrics_client()` (da Task PRE-0.7.4) para salvar dados de métricas, usando o Token de ID do usuário para autenticar a escrita no caminho `user_metrics/{uid}/...`
- Tarefa 1.5.3 (Lógica Backend/Cliente): Após cada análise LLM, coletar dados relevantes e chamar `save_metrics_client` com o Token de ID do usuário e seu uid.
- Tarefa 1.5.4 (UI): UI para feedback do usuário.
- Tarefa 1.5.5 (Lógica Backend/Cliente): Registrar o feedback do usuário como parte da métrica, usando `save_metrics_client`.
- Marco de Teste 1.5: Métricas de uso são registradas no Firestore sob o uid do usuário (`user_metrics/{uid}/...`) usando o Token de ID. Regras de Segurança garantem isolamento.
- **Módulo: `src.flet_ui` (Frontend - Interface Principal)**
 - Objetivo 1.6: Desenvolver a interface gráfica principal para interação com a análise.
 - [CONCLUÍDO - Estrutura] Tarefa 1.6.1: Utilizar `src.flet_ui.layout` (`AppBar`, `NavigationRail/Drawer`) e `src.flet_ui.router` para a estrutura da aplicação.
 - Tarefa 1.6.2: Criar view principal para:
 - Upload de PDF (usando `ft.FilePicker` ou o `ManagedFilePicker` se complexidade de upload web for um problema).
 - Disparar o processo de análise (que utiliza `PDFDocumentAnalyzer` e `ai_orchestrator`).
 - Exibir resultados da LLM (ex: em `ft.TextField` multiline ou `ft.Markdown`).
 - Fornecer botão "Copiar Resultado".
 - Integrar controles de feedback da Task 1.5.4.
 - Tarefa 1.6.3: Mostrar indicadores de progresso/carregamento durante o processamento do PDF e chamada LLM (usando `show_loading_overlay` de `components.py`).
 - Marco de Teste 1.6: Usuário logado consegue subir um PDF, o sistema processa, chama a LLM, exibe o resultado, permite copiar e dar feedback. Indicadores de carregamento são exibidos.

ITENS acrescentados:

- Upload de múltiplos arquivos, com reordenação;
- Containers de informações de metadados do processamento de PDF(s) e metadados de resposta do LLM;
- Drawer de configurações, à direita;
- Personalização do container de resposta da LLM com campos específicos conforme estrutura esperada (pendente).

- **Módulo: `src.core.doc_generator` (Backend - Exportação)**
 - Objetivo 1.7: Implementar funcionalidade de exportação simples para `.docx`.
 - Tarefa 1.7.1: Adicionar `python-docx` ao `pyproject.toml`.
 - Tarefa 1.7.2: Em `doc_generator.py`, implementar função que recebe o texto da resposta da LLM e o salva em um novo arquivo `.docx`.

- Tarefa 1.7.3 (UI): Adicionar botão "Exportar para DOCX" na view de resultados, que chama a função da Task 1.7.2 e usa `ft.FilePicker` para o usuário escolher onde salvar o arquivo.
 - Marco de Teste 1.7: Após a análise, o usuário pode clicar em "Exportar" e salvar um arquivo `.docx` contendo o resultado da LLM.
 - **Módulo: Empacotamento e Documentação Inicial**
 - Objetivo 1.8: Preparar a versão 0.1 para distribuição inicial.
 - Tarefa 1.8.1: Finalizar e testar o arquivo `.spec` do PyInstaller para incluir todos os assets, dados e dependências corretamente.
 - Tarefa 1.8.2: Gerar o executável final da v0.1.
 - Tarefa 1.8.3: Escrever um `README.md` básico com instruções de instalação, execução e configuração inicial (como adicionar a chave API OpenAI).
 - Tarefa 1.8.4: Criar contas no Firebase Authentication para os usuários iniciais.
 - Tarefa 1.8.5: Distribuir o pacote e as instruções para os usuários iniciais.
 - Marco de Teste 1.8: Pacote v0.1 é construído com sucesso, documentação `README` está disponível. Usuários iniciais conseguem instalar, configurar e usar as funcionalidades básicas da aplicação.
-

FASE 2: VERSÃO 1.0 – EXPANSÃO DE FUNCIONALIDADES E RAG

Objetivo Principal da Fase: Aprimorar a flexibilidade com múltiplos LLMs, implementar RAG e oferecer opções de exportação customizáveis.

- **Módulo: LLM & API Management (Backend: `ai_orchestrator`, `firebase_manager`; Frontend: `flet_ui`)**
 - Objetivo 2.1: Suportar múltiplos LLMs, modelos e chaves de API.
 - Tarefa 2.1.1 (Backend): Adaptar `FbManagerFirestore.save/get_user_api_key` para permitir múltiplas chaves por usuário, identificadas por um nome de serviço (ex: "openai_pessoal", "azure_corp").
 - Tarefa 2.1.2 (Backend): Em `ai_orchestrator`, generalizar a lógica de chamada LLM para suportar diferentes SDKs/APIs (OpenAI, Azure, Anthropic, etc.) com base na seleção do usuário.
 - Tarefa 2.1.3 (UI): Modificar a UI de Configurações para permitir adicionar/editar/remover múltiplas chaves API, associando um nome e o tipo de serviço.
 - Tarefa 2.1.4 (UI): Adicionar `ft.Dropdown` na UI principal para o usuário selecionar qual chave/serviço/modelo LLM deseja usar para a análise.
 - Tarefa 2.1.5 (Backend): Armazenar lista de LLMs/modelos suportados e suas configurações (ex: nome para display, endpoint se necessário) no `Firestore` para fácil atualização.
 - Marco de Teste 2.1: Usuário pode configurar múltiplas chaves API para diferentes serviços (ex: OpenAI, Azure OpenAI). Usuário pode selecionar na UI principal qual combinação de chave/serviço/modelo usar. A análise é realizada pelo LLM escolhido.
- **Módulo: RAG (Backend: `pdf_processor`, `ai_orchestrator`; Bibliotecas: `sentence-transformers`, `faiss-cpu`)**

- Objetivo 2.2: Implementar Retrieval Augmented Generation (RAG) "on the fly".
 - Tarefa 2.2.1: Adicionar sentence-transformers e faiss-cpu (ou alternativa leve como chromadb em modo in-memory) ao pyproject.toml.
 - Tarefa 2.2.2: Implementar download e cache local do modelo de embedding escolhido (ex: all-MiniLM-L6-v2) na primeira utilização.
 - Tarefa 2.2.3 (Backend): No pdf_processor ou ai_orchestrator, após extração e chunking inteligente do texto do PDF, gerar embeddings para os chunks.
 - Tarefa 2.2.4 (Backend): Criar um índice vetorial (FAISS ou similar) em memória com esses embeddings.
 - Tarefa 2.2.5 (Backend): Implementar função de busca no índice vetorial para encontrar os chunks mais relevantes para uma dada query (pergunta do usuário ou prompt interno).
 - Tarefa 2.2.6 (Backend): Adaptar a cadeia LangChain (ou lógica em ai_orchestrator) para:
 - Formular uma query interna baseada na tarefa de análise.
 - Buscar chunks relevantes usando o RAG.
 - Injetar os chunks relevantes como contexto no prompt final enviado ao LLM.
- Marco de Teste 2.2: Análise de documentos mais longos ou que exigem informações de múltiplas partes do texto produz resultados mais contextuais e precisos. É possível verificar (via logs ou debug) que o RAG está recuperando e injetando contexto relevante.
- **Módulo: Prompt Engineering & Token Management**
(Backend: ai_orchestrator, firebase_manager; Frontend: flet_ui)
 - Objetivo 2.3: Implementar múltiplos prompts com chunking e limites de tokens configuráveis.
 - Tarefa 2.3.1 (Backend): Desenvolver estratégia de chunking de texto otimizada para diferentes tipos de prompts/atributos.
 - Tarefa 2.3.2 (Backend): Definir estrutura no Firestore (via FbManagerFirestore) para armazenar múltiplos prompts (ex: por usuário ou globais, categorizados por tarefa/atributo). Incluir opção para prompts padrão.
 - Tarefa 2.3.3 (Backend): Na lógica de chamada LLM, selecionar o prompt apropriado (customizado do usuário ou padrão) para cada chunk ou atributo desejado.
 - Tarefa 2.3.4 (Backend): Armazenar limites de tokens (ex: por chamada, diário, mensal) por usuário no Firestore, configuráveis pelo administrador.
 - Tarefa 2.3.5 (Backend): Antes de cada chamada LLM, verificar o uso de tokens do usuário contra seus limites.
 - Tarefa 2.3.6 (UI): Na UI de Configurações (ou área admin), permitir visualização/edição dos prompts (se aplicável ao perfil).
 - Tarefa 2.3.7 (UI): Exibir notificação ao usuário se um limite de tokens for atingido, impedindo novas chamadas.
 - Marco de Teste 2.3: Diferentes atributos de um documento podem ser extraídos com prompts específicos, melhorando a precisão. Limites de tokens são respeitados, bloqueando chamadas quando excedidos. Prompts podem ser atualizados via Firestore sem re-deploy da aplicação.
- **Módulo: Exportação Avançada (Backend: doc_generator; Frontend: flet_ui)**
 - Objetivo 2.4: Implementar exportação em templates .docx customizáveis.

- Tarefa 2.4.1 (UI): Interface para o usuário fazer upload ou selecionar templates .docx pré-definidos (armazenados localmente ou no Firebase Storage).
- Tarefa 2.4.2 (Backend): Em doc_generator, implementar lógica para ler um template .docx, identificar placeholders (ex: {{NOME_PROCESSO}}, {{RESUMO_LLM}}).
- Tarefa 2.4.3 (Backend): Mapear os resultados da análise LLM (e metadados do PDF) para os placeholders e gerar o documento final preenchido.
- Tarefa 2.4.4 (UI): Botão "Exportar com Template" que permite ao usuário selecionar o template e salvar o arquivo gerado.
- Marco de Teste 2.4: Usuário pode selecionar um template DOCX, e a aplicação gera um novo DOCX com os dados da análise preenchidos nos locais corretos do template.
- **Módulo: Interação com PDF (Frontend: flet_ui; Backend: RAG da Task 2.2)**
 - Objetivo 2.5: Interface adicional estilo "Chat with your PDF".
 - Tarefa 2.5.1 (UI): Criar nova view/seção na UI com um campo de chat e área de exibição de respostas.
 - Tarefa 2.5.2 (UI/Backend): Quando o usuário envia uma pergunta:
 - A pergunta é enviada para o backend.
 - O backend usa o RAG (Task 2.2 - índice vetorial do PDF atual) para encontrar chunks relevantes.
 - Um prompt é construído com a pergunta do usuário e o contexto recuperado.
 - O prompt é enviado ao LLM selecionado.
 - A resposta do LLM é exibida na UI de chat.
 - Marco de Teste 2.5: Usuário pode fazer perguntas em linguagem natural sobre o conteúdo do PDF carregado e receber respostas contextuais geradas pela LLM com o auxílio do RAG.

FASE 3: VERSÃO 1.5 – SEGURANÇA E ATUALIZAÇÕES

Objetivo Principal da Fase: Incrementar a segurança dos dados processados e facilitar a distribuição de novas versões da aplicação.

- **Módulo: src.security.anonymizer (Backend - Segurança)**
 - Objetivo 3.1: Implementar sistema de anonimização de dados sensíveis.
 - Tarefa 3.1.1: Adicionar dependências de NER (ex: spacy e modelo de linguagem PT, ou presidio-analyzer) ao pyproject.toml.
 - Tarefa 3.1.2: Em anonymizer.py, implementar lógica para carregar modelo NER e identificar entidades sensíveis (nomes, CPFs, emails, etc.) no texto extraído do PDF.
 - Tarefa 3.1.3: Implementar mascaramento ou substituição das entidades identificadas (ex: [PESSOA_1], [EMAIL_1]). Manter um mapa de anonimização em memória (placeholder -> valor original) durante a sessão de processamento.
 - Tarefa 3.1.4: Integrar o processo de anonimização no fluxo principal: após a extração de texto e antes do envio para a LLM. O texto enviado à LLM deve ser o anonimizado.
 - Tarefa 3.1.5 (Opcional): Implementar lógica para desanonimizar a resposta da LLM usando o mapa em memória, antes de exibir ao usuário, se os placeholders aparecerem na resposta e a re-identificação for desejada.
 - Tarefa 3.1.6 (UI): Adicionar uma opção (checkbox ou configuração) para o usuário habilitar/desabilitar a anonimização, se aplicável.

- Marco de Teste 3.1: Textos contendo dados pessoais identificáveis são processados, e o payload enviado à LLM contém os dados anonimizados (placeholders). O resultado exibido ao usuário pode estar anonimizado ou re-identificado, conforme a configuração.
 - **Módulo: updater (Novo Módulo - Utilitário Desktop)**
 - Objetivo 3.2: Desenvolver e implementar um atualizador automático (Updater.exe).
 - Tarefa 3.2.1: Criar um novo script Python (updater_logic.py) que:
 - Recebe como argumentos a URL do novo pacote e a versão.
 - Baixa o pacote de atualização (ex: ZIP) de forma segura.
 - (Recomendado) Verifica o checksum do pacote baixado contra um valor esperado.
 - (Opcional) Faz backup da versão atual da aplicação.
 - Extrai e substitui os arquivos da aplicação principal.
 - Tenta reiniciar a aplicação principal.
 - Tarefa 3.2.2: Empacotar updater_logic.py usando PyInstaller para criar Updater.exe.
 - Tarefa 3.2.3 (Firebase): Configurar uma área no Firebase Storage para hospedar os pacotes de atualização (ex: app_v1.5.1.zip).
 - Tarefa 3.2.4 (Firebase): Criar um documento/arquivo no Firestore ou Storage (ex: version_info.json) contendo a versão mais recente, URL do pacote e checksum.
 - Tarefa 3.2.5 (Aplicação Principal): Na inicialização (run.py ou flet_ui/app.py), adicionar lógica para:
 - Consultar version_info.json no Firebase.
 - Comparar com a versão atual da aplicação.
 - Se uma nova versão estiver disponível, notificar o usuário (ex: com show_confirmation_dialog).
 - Se o usuário confirmar, fechar a aplicação principal e executar Updater.exe passando os parâmetros necessários.
 - Marco de Teste 3.2: Aplicação principal detecta uma nova versão disponível no Firebase, notifica o usuário. Se confirmado, o Updater.exe é lançado, baixa o novo pacote, substitui os arquivos e a aplicação é atualizada e reiniciada com a nova versão.
-

FASE 4: VERSÃO 2.0 – INTEGRAÇÃO COM RPA

Objetivo Principal da Fase: Permitir que processos de RPA interajam com as funcionalidades da aplicação.

- **Módulo: rpa_integration (Novo Módulo - Backend/Interface)**
 - Objetivo 4.1: Desenvolver e documentar uma interface para integração com scripts de RPA.
 - Tarefa 4.1.1: Avaliar e decidir a melhor abordagem para a interface de integração:
 - Opção A: API HTTP local exposta pela aplicação Python (ex: usando Flask/FastAPI em uma thread separada, se viável com Flet).
 - Opção B: Firebase Functions que atuam como um gateway, chamadas pelos scripts RPA e que interagem com o backend da aplicação (ou diretamente com LLMs se for um fluxo simplificado).
 - Opção C: Interface baseada em arquivos/CLI (menos robusta, mas simples).
 - Tarefa 4.1.2: Se API HTTP/Functions: Definir os endpoints (ex: /analyze_pdf), métodos (POST), formato dos requests (ex: multipart/form-data para PDF, JSON para parâmetros) e responses (JSON com resultados).

- Tarefa 4.1.3: Implementar a lógica da interface escolhida no novo módulo `rpa_integration`. Isso incluirá receber o PDF, invocar o `PDFDocumentAnalyzer` e `ai_orchestrator`, e retornar os resultados.
- Tarefa 4.1.4: Implementar mecanismos de autenticação e autorização para os scripts RPA (ex: API keys dedicadas, tokens de serviço se Firebase Functions).
- Tarefa 4.1.5: Criar documentação clara da API/interface para os desenvolvedores de RPA (endpoints, payloads, autenticação, exemplos).
- Marco de Teste 4.1: Um script de RPA de teste consegue enviar um PDF para a interface de integração, a aplicação processa o documento e retorna os resultados da análise para o script RPA. A autenticação funciona corretamente. A documentação da API está completa.

FASE 5: VERSÃO 2.5 – SUPORTE A LLMS LOCAIS (OFFLINE)

Objetivo Principal da Fase: Oferecer a opção de processamento totalmente offline, utilizando LLMs executadas localmente.

- **Módulo: `local_llms` (Novo Módulo - Backend/Interface)**

- Objetivo 5.1: Integrar suporte para execução de LLMs locais.
 - Tarefa 5.1.1: Pesquisar e selecionar frameworks para LLMs locais (ex: Llama.cpp, Ollama, GPT4All) e modelos compatíveis (quantizados para CPU, ou com suporte a GPU se for um requisito). Considerar facilidade de empacotamento e uso.
 - Tarefa 5.1.2: Implementar lógica no módulo `local_llms` para:
 - Gerenciar o download/instalação de modelos LLM locais (se não forem pré-empacotados).
 - Carregar e executar inferências com os modelos locais selecionados.
 - Expor uma interface unificada (similar à usada para LLMs em nuvem) para o `ai_orchestrator`.
 - Tarefa 5.1.3: Adaptar `ai_orchestrator` para poder direcionar as requisições para os LLMs locais configurados, além das LLMs em nuvem.
 - Tarefa 5.1.4: Avaliar e documentar os requisitos de hardware (CPU, RAM, GPU opcional, espaço em disco) para a execução de LLMs locais.
- Marco de Teste 5.1: Aplicação consegue carregar um modelo LLM local, processar um PDF e realizar a análise usando esse modelo local sem conectividade com a internet. Os resultados são comparáveis (em qualidade, dentro do esperado para o modelo local) aos de LLMs na nuvem.

- **Módulo: `src.flet_ui` (Frontend - Configuração LLM Local)**

- Objetivo 5.2: Desenvolver interface para gerenciamento e seleção de LLMs locais.
 - Tarefa 5.2.1 (UI): Criar nova seção na UI de Configurações para LLMs Locais, permitindo:
 - Listar modelos locais disponíveis/compatíveis.
 - Iniciar download/configuração de novos modelos locais.
 - Verificar status dos modelos locais.
 - Tarefa 5.2.2 (UI): Modificar o seletor de LLM/modelo (da Task 2.1.4) para incluir a opção de "LLM Local" e permitir a seleção do modelo local específico.

- Marco de Teste 5.2: Usuário pode visualizar, configurar (se necessário) e selecionar um LLM local através da interface. A aplicação usa o LLM local selecionado para as análises.

FASE 6: VERSÃO 3.0 – AGENTES E MODEL CONTEXT PROTOCOL (MCP)

Objetivo Principal da Fase: Evoluir a arquitetura da aplicação para um modelo mais sofisticado baseado em agentes de IA e o Model Context Protocol, permitindo maior autonomia, flexibilidade e capacidade de lidar com tarefas mais complexas e interativas.

- **Módulo: agents_pipelines (Novo Módulo - Core Backend)**

- Objetivo 6.1: Re-arquitetar a lógica de IA utilizando Agentes e o Model Context Protocol.
 - Tarefa 6.1.1: Pesquisar e aprofundar o entendimento sobre frameworks de Agentes (ex: LangChain Agents, AutoGen) e as especificações/implicações do Model Context Protocol (MCP).
 - Tarefa 6.1.2: Redesenhar o fluxo de análise de documentos como uma colaboração entre múltiplos agentes especializados (ex: Agente de Extração, Agente de RAG, Agente de Análise, Agente de Formatação).
 - Tarefa 6.1.3: Implementar os agentes definidos, utilizando ferramentas e abstrações apropriadas (LangChain, etc.).
 - Tarefa 6.1.4: Implementar a comunicação e troca de contexto entre agentes seguindo os princípios do Model Context Protocol (MCP), garantindo que o estado e o conhecimento sejam gerenciados e passados de forma eficiente e padronizada.
 - Tarefa 6.1.5: Adaptar os módulos existentes (pdf_processor, ai_orchestrator se partes dele forem reutilizadas como ferramentas para os agentes) para se integrarem à nova arquitetura baseada em agentes e MCP.
- Marco de Teste 6.1: A análise de documentos é realizada por um sistema de agentes que colaboram e trocam informações via MCP. O sistema demonstra maior flexibilidade para lidar com variações nas tarefas de análise e potencialmente suporta fluxos mais complexos (ex: múltiplas rodadas de perguntas, sumarização iterativa).

- **Módulo: Monitoramento, Operações e UI para Agentes**

- Objetivo 6.2: Adaptar monitoramento, operações e a UI para a nova arquitetura.
 - Tarefa 6.2.1 (Backend): Desenvolver mecanismos de logging e monitoramento específicos para o comportamento dos agentes, o fluxo de contexto via MCP e o desempenho de cada componente do pipeline.
 - Tarefa 6.2.2 (Backend): Definir e implementar estratégias para gerenciamento de estado das conversas/análises com os agentes e configuração dos pipelines de agentes.
 - Tarefa 6.2.3 (UI): (Opcional, dependendo da complexidade) Avaliar a necessidade de uma UI para visualizar o status dos agentes, o progresso em pipelines complexos ou para configurar o comportamento dos agentes/pipelines.
- Marco de Teste 6.2: Logs detalhados sobre a execução dos agentes e o fluxo MCP estão disponíveis. É possível monitorar o estado e o desempenho do sistema de agentes. A UI (se modificada) reflete a nova capacidade de interação ou configuração.

Registros de Desenvolvimento

Troubleshooting Log

1. Armazenamento Seguro de Credenciais Grandes (Ex: Chave Firebase JSON)

- **Problema:** Tentativa de armazenar diretamente o conteúdo completo de um arquivo JSON (como chave de serviço Firebase) no keyring resulta em erro no backend do Windows Credential Manager ((1783, 'CredWrite', 'O fragmento de código recebeu dados incorretos')) devido a limites de tamanho.
- **Solução:** Abordagem Híbrida (Keyring + Arquivo Criptografado):
 1. **Gerar Chave de Criptografia:** Utilizar a biblioteca cryptography (especificamente Fernet) para gerar uma chave simétrica forte (Fernet.generate_key()). Esta chave é pequena (bytes).
 2. **Armazenar Chave no Keyring:** Guardar *apenas* a chave Fernet (convertida para string, ex: key.decode()) no keyring (keyring.set_password(SERVICE_NAME, KEY_USERNAME, fernet_key_string)). O keyring protege esta chave pequena usando os mecanismos seguros do OS.
 3. **Criptografar Conteúdo:** Ler o conteúdo original do arquivo JSON. Criptografar este conteúdo usando a instância Fernet (fernet.encrypt(json_content.encode())) criada com a chave recuperada do keyring.

4. **Salvar Arquivo Criptografado:** Salvar os *bytes criptografados* resultantes em um arquivo local persistente (ex: `credenciais.enc`) em um diretório de dados da aplicação (ex: `%APPDATA%\NomeApp` no Windows, `~/.config/NomeApp` no Linux, `~/Library/Application Support/NomeApp` no macOS). Usar `os.path.join` e `os.getenv/os.path.expanduser` para portabilidade.
 5. **Leitura:** Em execuções subsequentes, o fluxo é inverso: ler chave do keyring, ler arquivo `.enc`, descriptografar com Fernet, usar JSON.
- **Benefícios:** Contorna o limite de tamanho do keyring enquanto aproveita sua segurança para proteger a chave de criptografia principal. O conteúdo sensível grande fica armazenado de forma criptografada no disco.
 - **Dependências:** keyring, cryptography.

2. Upload de Arquivos no Flet Web (AppView.WEB_BROWSER)

- **Problema:** O controle `ft.FilePicker` no modo Web não fornece o caminho local (`path=None`) por razões de segurança do navegador. É necessário implementar um fluxo de upload para obter o conteúdo do arquivo no backend Python.
- **Solução (Fluxo Atualizado Flet 0.21.0+):**
 1. **Chave Secreta:** É **obrigatório** definir a variável de ambiente `FLET_SECRET_KEY` antes de iniciar a aplicação (`os.environ["FLET_SECRET_KEY"] = "sua_chave_segura"`). Isso é usado para assinar as URLs de upload. Para desenvolvimento, pode-se definir no script; em produção, usar variáveis de ambiente do sistema/deploy.
 2. **Diretório de Upload:** Definir um diretório no servidor onde os arquivos carregados serão temporariamente salvos. Passar o caminho (preferencialmente absoluto) para `ft.app(..., upload_dir=CAMINHO_ABS_UPLOAD)`. É recomendável limpar este diretório na inicialização da aplicação (`shutil.rmtree`, `os.makedirs`).
- 3. **Seleção (FilePicker.on_result):**
 - No callback, obter o nome do arquivo (`e.files[0].name`).
 - Gerar a URL de upload assinada: `upload_url = page.get_upload_url(file_name, expires=...)`.
 - Preparar a lista de upload: `upload_list = [ft.FilePickerUploadFile(name=file_name, upload_url=upload_url)]`.
 - (Opcional/Recomendado): Limpar qualquer arquivo com o mesmo nome no `upload_dir` antes de iniciar um novo upload para evitar conflitos (`os.remove`).
 - **Iniciar Upload:** Chamar o método do controle `FilePicker`: `file_picker.upload(files=upload_list)`.
- 4. **Monitoramento (FilePicker.on_upload):**
 - Atribuir uma função de callback para `FilePicker` (`on_upload=...`).

- Este callback recebe eventos `ft.FilePickerUploadEvent` com `file_name`, `progress` (0.0 a 1.0) e `error`.
- Atualizar a UI (ex: `ft.ProgressBar`) com `e.progress`.
- Tratar `e.error`.
- **Ao Concluir (`e.progress == 1.0` e `not e.error`):**
 - Construir o caminho esperado do arquivo no servidor: `uploaded_path = os.path.join(CAMINHO_ABS_UPLOAD, e.file_name)`.
 - **Tratar Race Condition:** Implementar um loop de tentativa (ex: 5 tentativas com `time.sleep(0.3)` entre elas) verificando `os.path.exists(uploaded_path)` antes de prosseguir, pois o arquivo pode não estar imediatamente visível/pronto no sistema de arquivos após o evento `progress=1.0`. Logar `os.listdir(CAMINHO_ABS_UPLOAD)` na primeira tentativa ajuda a depurar.
 - Se o arquivo for encontrado, chamar a função de processamento do backend passando `uploaded_path`.

5. Processamento Backend:

- A função recebe o caminho absoluto do arquivo temporário carregado.
- Realiza as operações necessárias (ler, validar, criptografar, etc.).
- **Limpeza: Fundamental** remover o arquivo temporário de `CAMINHO_ABS_UPLOAD` usando `os.remove(uploaded_path)` dentro de um bloco `finally` para garantir a limpeza mesmo em caso de erro no processamento.

- **Componentes**

Chave: `ft.FilePicker`, `os.environ["FLET_SECRET_KEY"]`, `page.get_upload_url`, `ft.FilePickerUploadFile`, `file_picker.upload`, `FilePicker(on_upload=...)`, `ft.app(upload_dir=...)`, Lógica de Retentativa (`os.path.exists`, `time.sleep`), `os.remove` para limpeza.

- **Dependências:** `flet`, `time`.

3. Redução da Verbosidade dos Logs (Despoluição)

- **Problema:** Ao definir o nível de log geral para `DEBUG` (`logging.basicConfig(level=logging.DEBUG)`), a saída do console fica poluída com mensagens `DEBUG` detalhadas de bibliotecas internas (`Flet`, `Uvicorn`, `Websockets`, etc.).
- **Solução:** Configuração de Logging Específica por Logger:
 1. **Configurar Raiz:** Manter (ou definir) o nível desejado para o logger raiz, que captura os logs do seu próprio código: `logging.basicConfig(level=logging.DEBUG, format=...)`.

2. **Identificar Loggers Verbosos:** Observar os nomes dos loggers que aparecem nas mensagens DEBUG indesejadas (ex: `flet_core`, `flet_web`, `websockets`, `uvicorn`, `uvicorn.access`). O nome aparece no formato do log se `%(name)s` for incluído.
3. **Aumentar Nível de Loggers Específicos:** Para cada logger identificado, obter sua instância e definir um nível mais alto (ex: INFO, WARNING, ERROR) para suprimir mensagens menos importantes:

```
logging.getLogger("flet_core").setLevel(logging.WARNING)
logging.getLogger("websockets").setLevel(logging.WARNING)
logging.getLogger("uvicorn.access").setLevel(logging.WARNING)
```
- **Resultado:** A saída do console conterá as mensagens DEBUG do seu código, mas apenas mensagens INFO ou mais graves (WARNING, ERROR) das bibliotecas configuradas, tornando o log mais limpo e focado.
- **Dependências:** `logging`.

4. Erro `AttributeError: 'NoneType' object has no attribute 'isatty'` ao Iniciar `.exe` Flet Web

- **Problema:** Ao executar o `.exe` compilado com PyInstaller (especialmente sem um console visível ou no modo `--windowed`), a aplicação falha na inicialização com `AttributeError: 'NoneType' object has no attribute 'isatty'` seguido por `ValueError: Unable to configure formatter 'default'`. Isso ocorre porque o servidor ASGI Uvicorn, usado internamente pelo Flet no modo `WEB_BROWSER`, tenta configurar seu logger padrão ('default') e verifica se `sys.stderr/stdout` é um terminal interativo (TTY) usando `isatty()`. No ambiente do `.exe`, esses streams podem ser `None` ou inválidos, causando a falha. Tentativas de configurar o logger raiz manualmente ou fazer monkey patch de `sys.stderr` não foram suficientes, pois Uvicorn ainda tentava configurar seu formatter interno.
- **Solução:** Monkey Patching Direto do Formatter do Uvicorn:
 1. Importar `DefaultFormatter` de `uvicorn.logging` (e `click`, uma dependência implícita).
 2. Criar uma subclasse (ex: `SafeUvicornDefaultFormatter`) que herda de `DefaultFormatter`.
 3. Na `__init__` da subclasse, forçar o argumento `use_colors=False` ao chamar `super().__init__(*args, **kwargs)`. Isso desabilita a lógica que causa a verificação `isatty`. Incluir tratamento de erro básico (`try...except TypeError`) na chamada `super().__init__` para compatibilidade entre versões do Uvicorn.
 4. **Antes** de chamar `ft.app()`, substituir a classe original no módulo Uvicorn pela subclasse segura: `uvicorn.logging.DefaultFormatter = SafeUvicornDefaultFormatter`.
 5. Garantir que `uvicorn` e `click` sejam incluídos nos `hiddenimports` do PyInstaller (no arquivo `.spec`) para que estejam disponíveis no `.exe`.

- **Resultado:** O Uvicorn consegue inicializar seu logger sem erros, pois a verificação isatty problemática é evitada.
- **Dependências/Componentes:** uvicorn, click, Edição do .spec (hiddenimports), Monkey Patching.

5. Erro FileNotFoundError para flet_web/web/index.html (ou similar) no .exe Flet Web

- **Problema:** Ao executar o .exe no modo WEB_BROWSER, ocorre FileNotFoundError ou Exception: Web root path not found: ..._MEIxxxxx\\flet_web\\web porque o PyInstaller não empacotou automaticamente os arquivos estáticos (HTML, JS, assets) necessários para a interface web, que residem na subpasta web do pacote flet_web instalado.
- **Solução:** Adicionar Explicitamente os Dados do Pacote flet_web via Arquivo .spec:
 1. Garantir que import flet_web e import os estejam no **topo** do arquivo .spec.
 2. Calcular o caminho absoluto para a pasta flet_web instalada: flet_web_path = os.path.dirname(flet_web.__file__).
 3. Na seção Analysis, localizar a lista datas=[] (ou criá-la).
 4. Adicionar uma tupla especificando a origem (pasta web dentro de flet_web) e o destino relativo correto dentro do pacote: datas+=[(os.path.join(flet_web_path, 'web'), 'flet_web')].
 - **Atenção ao Destino:** O destino deve ser 'flet_web', para que o PyInstaller crie a estrutura _internal/flet_web/web/index.html que o Flet espera (e não _internal/flet_web/web/web/index.html).
 5. Recompilar usando o arquivo .spec: pyinstaller seu_app.spec.
- **Resultado:** PyInstaller inclui a pasta flet_web/web com seus conteúdos na estrutura correta dentro do .exe (ou diretório _internal no modo --onedir), permitindo que o Flet sirva a interface web corretamente.
- **Dependências/Componentes:** Edição do .spec (datas), flet_web, os.

6. A interface do usuário (UI) congela ou não renderiza um ft.AlertDialog modal quando este contém ou ativa um ft.FilePicker, especificamente durante a fase de inicialização da aplicação Flet (logo após a conexão do cliente na função main). A execução do código Python para exibir o diálogo (page.update()) ocorre sem erros, mas o diálogo não aparece visualmente.

Contexto:

Necessidade de solicitar um arquivo de configuração JSON ao usuário via AlertDialog + FilePicker se o arquivo não for encontrado na inicialização.

Observações:

- AlertDialog *sem* FilePicker funciona corretamente na inicialização.

- AlertDialog com FilePicker funciona corretamente quando acionado por interação do usuário *após* a inicialização completa da UI principal.
- O problema ocorre devido a conflitos de temporização/atualização da UI ao manipular `page.controls`, `page.overlay` e exibir diálogos modais em rápida sucessão durante o ciclo de vida inicial sensível do Flet.

Abordagens Ineficazes (Durante Inicialização):

1. Exibição direta do diálogo (com FilePicker associado) após verificação de arquivo.
2. Uso de `threading.Timer` para atrasar a exibição do diálogo após limpeza/placeholder da UI.
3. Adição do FilePicker ao `page.overlay` apenas no `on_click` do botão dentro do diálogo.
4. Uso de APIs de diálogo obsoletas (`page.show_dialog`).

Solução Implementada (Padrão Recomendado Flet >= 0.21.0):

1. **async main:** Tornar a função `main` do `ft.app` assíncrona (`async def main(page: ft.Page):`).
2. **Adição Antecipada do FilePicker:** Instanciar `ft.FilePicker` e adicioná-lo ao `page.overlay` *imediatamente* dentro da função `main`, antes de criar o controlador principal da UI. Passar a instância do FilePicker para o controlador.
3. **Controlador (AppController):**
 - Receber e armazenar a instância do FilePicker.
 - Definir o `FilePicker.on_result` para um método interno do controlador.
 - Usar `async def` para o método de fluxo de inicialização (`start_initialization_flow`).
4. **Atraso Assíncrono:** Dentro do fluxo de inicialização do controlador, usar `await asyncio.sleep(0.3)` *antes* de verificar a necessidade do diálogo, permitindo que a UI inicial e o overlay se estabilizem.
5. **Gerenciamento do AlertDialog via Controlador:**
 - Após o `asyncio.sleep` e a verificação, se o diálogo for necessário:
 - Criar a instância do `ft.AlertDialog` (usando uma função auxiliar que recebe o `file_picker`).
 - Adicionar a instância do diálogo ao `page.overlay` (idealmente apenas uma vez).
 - Definir `dialog.open = True`.
 - Chamar `page.update()` (ou `await page.update_async()`).
 - O botão dentro do diálogo simplesmente chama `file_picker.pick_files()`.
6. **Tratamento do Resultado (FilePicker.on_result no Controlador):**
 - Definir `dialog.open = False`.
 - Chamar `page.update()` (ou `await page.update_async()`) para fechar o diálogo.

- Processar o arquivo selecionado.
- Se o processamento for bem-sucedido, remover o diálogo do `page.overlay` (`page.overlay.remove(dialog)`) e limpar a referência.

Resultado:

Esta abordagem com adição antecipada do overlay, delay assíncrono e gerenciamento explícito do estado open do diálogo no overlay resolve o congelamento e permite a exibição confiável do diálogo com `FilePicker` durante a inicialização.

Adendo: Correção de Depreciações (`ft.icons`, `ft.colors`)

Durante a análise e correção do problema principal, também foi observado e corrigido o uso de enums deprecados. As referências a ícones via `ft.icons.<NOME_ICONE>` foram substituídas pela sintaxe recomendada `ft.Icons.<NOME_ICONE>`. Similarmente, referências a cores via `ft.colors.<NOME_COR>` foram atualizadas para `ft.Colors.<NOME_COR>`. Essa atualização garante a compatibilidade com versões futuras do Flet ($\geq 0.25.0$) onde os enums antigos serão removidos.

****7. Upload de Arquivo no Modo Web "Trava" ou Não Inicia Corretamente na Primeira Tentativa****

* ****Sintoma:**** Ao tentar fazer upload de um arquivo no modo ``AppView.WEB_BROWSER`` usando ``ft.FilePicker``, a interface do usuário pode parecer "travada" ou o processo de upload não é iniciado (o callback ``FilePicker.on_upload`` não é acionado no backend Python). O log da aplicação pode parar após a mensagem indicando que o "Comando de upload foi enviado ao Flet", mas nenhum progresso ou conclusão é reportado. Comentar chamadas a ``show_snackbar`` (que internamente usa ``page.update()``) próximas à chamada ``file_picker.upload()`` pode, às vezes, fazer o upload funcionar.

* ****Causa Provável:**** Conflito de temporização na atualização da UI e no processamento de eventos assíncronos do Flet. Se ``page.update()`` (chamado direta ou indiretamente, por exemplo, via ``show_snackbar``) é executado muito próximo ou imediatamente antes de uma operação que também requer comunicação com o backend do Flet (como ``file_picker.upload()``), pode ocorrer um bloqueio ou atraso no processamento do evento de upload. O Flet precisa de "tempo de respiro" em seu event loop para lidar com ambas as atualizações da UI e o início de operações assíncronas.

* ****Solução Aplicada:**** Desacoplamento temporal usando ``threading.Timer`` no ``ManagedFilePicker``.

1. ****Feedback Imediato:**** No método ``_on_file_picker_result`` (após o usuário selecionar um arquivo no modo web):

* Um ``show_snackbar`` é chamado imediatamente para informar ao usuário que o envio está começando (ex: "Enviando 'arquivo.pdf...'"). Esta função já contém um ``page.update()``.

2. ****Upload Agendado:**** A chamada real ``self.file_picker.upload()`` ****não**** é executada imediatamente. Em vez disso, ela é encapsulada em uma função auxiliar (``_trigger_actual_upload``).

3. **Timer:** Um `threading.Timer` é usado para agendar a execução de `_trigger_actual_upload` após um pequeno delay (ex: 0.2 segundos).

4. **Execução Desacoplada:** O timer permite que o `page.update()` do `show_snackbar` seja completamente processado pelo Flet. Após o delay, a thread do timer executa `_trigger_actual_upload`, que então chama `self.file_picker.upload()`. Neste ponto, o Flet está mais propenso a processar corretamente o comando de upload e iniciar o fluxo de eventos `on_upload`.

* **Gerenciamento de Estado da UI (Botões):**

- * É crucial desabilitar os botões relevantes (ex: "Carregar PDF", "Analisar") *antes* de chamar `file_picker.pick_files()` e atualizar a UI (`page.update()`).
- * Os botões devem ser reabilitados no callback final (`on_upload_complete` na view), considerando todos os cenários: sucesso do upload, falha no upload, ou cancelamento da seleção pelo usuário.

* **Componentes Chave:** `ManagedFilePicker`, `ft.FilePicker.pick_files()`, `ft.FilePicker.upload()`, `FilePicker.on_result`, `FilePicker.on_upload`, `threading.Timer`, `page.update()`, `show_snackbar`.

* **Resultado:** A interface permanece responsiva. O feedback visual ("Enviando...") aparece, e o processo de upload é iniciado de forma mais confiável após o pequeno delay, permitindo que os eventos de `on_upload` sejam recebidos e processados pelo backend.

Outro Cenário: Um diálogo é fechado, e o callback (`on_select_callback`) tenta imediatamente atualizar a view principal.

Decorador no Callback:

```
def delay_ui_update_after_dialog_close(delay=0.1):
    def decorator(callback_func):
        @functools.wraps(callback_func)
        def wrapper(page, *args, **kwargs): # Supondo que o callback receba 'page'
            def delayed_execution():
                callback_func(page, *args, **kwargs)
                # O callback_func pode chamar show_snackbar ou page.update()

            # O diálogo em si já foi fechado visualmente antes deste wrapper ser chamado.
            # Este decorador apenas atrasa a lógica de negócios do callback.
            threading.Timer(delay, delayed_execution).start()
            return wrapper
        return decorator
```

Problema: O upload de arquivos via `ft.FilePicker` no modo web pode "travar" após o envio do comando, sem acionar o callback `on_upload` ou com atraso.

Causa: O Flet depende de WebSockets para comunicação, e a UI precisa de uma atualização explícita (`page.update()`) após `file_picker.upload()` para registrar e iniciar corretamente o processo.

Solução:

Atualização imediata da UI: Após `file_picker.upload([...])`, chamar `self.page.update()`.

Controle de feedback visual pela view: Delegar a exibição de status ao frontend e remover `show_snackbar` antes do upload.

Gerenciamento de estado dos botões: Desabilitar o botão de upload antes de `pick_files()` e reabilitá-lo via callback `on_upload_complete`.

Resultado: O upload torna-se confiável, garantindo resposta adequada da UI e execução correta dos eventos no backend Python.

Problema:

`AssertionError: [NomeDoControle] Control must be added to the page first` ou `AssertionError (com self.__uid is None)` durante a inicialização de views/componentes Flet complexos.

Recomendações Futuras (Prevenção):

Ao desenvolver novas views ou componentes Flet:

- 1) Evitar chamadas `.update()` em controles individuais ou no `self` dentro do `__init__` ou métodos de setup inicial.
- 2) Concentrar as atualizações de UI em chamadas `page.update()` no nível do router ou após a montagem completa do componente na árvore da página.
- 3) Para atualizações dinâmicas após a inicialização (ex: em resposta a eventos de usuário), o `.update()` nos controles específicos ou no `page.update()` continua sendo a abordagem correta, pois nesse ponto os controles já estão "vivos" na página.

Solução para Download de Arquivos (ex: DOCX) no Flet em Modo Web (AppView.WEB_BROWSER)

Para permitir que os usuários baixem arquivos gerados pela aplicação Flet quando esta está rodando no modo web, a abordagem recomendada é gerar o arquivo no servidor e, em seguida, instruir o navegador do cliente a acessá-lo através de uma URL que o servidor Flet possa servir.

1. Geração do Arquivo no Servidor:

- A lógica de backend da aplicação Python (ex: um `DocxExporter`) gera o arquivo desejado (ex: `relatorio.docx`) e o salva em um diretório no servidor. Este diretório é referido aqui como `DIR_ARQUIVOS_GERADOS`.

2. Configurar `assets_dir` no `ft.app()`:

- No script principal que inicia a aplicação Flet (ex: `run.py`), o caminho para `DIR_ARQUIVOS_GERADOS` (deve ser um caminho absoluto) precisa ser passado para o parâmetro `assets_dir` da função `ft.app()`.
- Isso instrui o servidor web embutido do Flet a servir os arquivos contidos em `DIR_ARQUIVOS_GERADOS_ABS` diretamente sob a URL raiz da aplicação.

3. Construir a URL de Download:

- Após o arquivo ser salvo no servidor (ex: `relatorio_final.docx` dentro de `DIR_ARQUIVOS_GERADOS_ABS`), a URL para acessá-lo (do ponto de vista do navegador) será simplesmente `/relatorio_final.docx`. O nome da pasta `DIR_ARQUIVOS_GERADOS_ABS` não faz parte da URL quando usado como `assets_dir`.

4. Iniciar o Download via `page.launch_url()`:

- Na lógica do Flet que lida com a ação de exportação/download, após o arquivo ser gerado no servidor, use `page.launch_url()`:
- `web_window_name="_self"` tentará iniciar o download na aba atual, enquanto `"_blank"` abrirá uma nova aba (que provavelmente fechará após o início do download).

5. Comportamento do Navegador:

- Quando o navegador recebe a instrução para ir para `/relatorio_final.docx`, ele faz uma requisição GET para o servidor Flet.
- O servidor Flet, devido à configuração `assets_dir`, localiza o arquivo e o serve. Para tipos de arquivo comuns como `.docx`, o servidor Flet geralmente envia um `Content-Type` apropriado (`application/vnd.openxmlformats-officedocument.wordprocessingml.document`).
- A maioria dos navegadores modernos, ao receberem este tipo de conteúdo, iniciará automaticamente o processo de download para a pasta de downloads padrão do usuário, sem exibir um diálogo de "Salvar Como" da aplicação Flet.

6. Limpeza de Arquivos (Importante):

- Como os arquivos estão sendo gerados no servidor, é crucial implementar uma estratégia para limpar o `DIR_ARQUIVOS_GERADOS` periodicamente para evitar o acúmulo de arquivos. Isso pode ser feito na inicialização da aplicação, através de uma tarefa agendada, ou removendo arquivos mais antigos que um determinado período.

Nota: Esta abordagem não utiliza `FilePicker.save_file()` para o modo web, pois este método é primariamente destinado a aplicações desktop e não fornece um diálogo "Salvar Como" nativo no navegador. A combinação de `assets_dir` e `page.launch_url()` é a forma idiomática no Flet para downloads iniciados pelo servidor em aplicações web.